UNIVERSIDAD DE CARABOBO
FACULTAD DE CIENCIAS Y TECNOLOGÍA
ESCUELA DE COMPUTACIÓN
CAMPUS BÁRBULA
ARQUITECTURA DEL COMPUTADOR

Proyecto Final 14: Impacto del Tamaño de Línea de Caché en el Rendimiento

Daniel Gutierrez CI: 32047059
Gabriel Diaz CI: 31161550

Bárbula, Octubre de 2026

1. El Problema

En la arquitectura de computadores, el diseño de la memoria caché es un factor determinante para mitigar la brecha de velocidad entre el procesador y la memoria principal. Uno de los parámetros de diseño más críticos es el **tamaño de la línea de caché** (o tamaño de bloque).

El problema central radica en el principio de *localidad espacial*. Aumentar el tamaño de la línea permite que, ante un fallo de caché, se traigan más datos adyacentes desde la memoria principal, previendo que el procesador los necesitará pronto. Sin embargo, un tamaño de línea excesivamente grande genera dos efectos negativos: primero, incrementa severamente la penalización por fallo (el tiempo que tarda el bus en transferir un bloque grande); segundo, reduce la cantidad total de líneas que caben en la caché, lo que provoca que datos útiles sean expulsados prematuramente, un fenómeno conocido como *contaminación de caché*.

El objetivo de este proyecto es simular y analizar empíricamente cómo la variación de este tamaño de línea afecta las métricas críticas del sistema, específicamente la Tasa de Aciertos (Hit Rate) y el Tiempo Medio de Acceso a Memoria (AMAT).

2. Técnicas Computacionales para su Solución

Para la simulación del hardware se desarrolló una herramienta en **C++11**, compilada mediante `GCC` en un entorno GNU/Linux. El modelo emplea programación orientada a objetos para encapsular la jerarquía de memoria y estructuras de datos dinámicas de la STL (`std::vector`) para representar los conjuntos y vías de la caché.

Las técnicas algorítmicas clave implementadas son:

- **Decodificación a nivel de Bits:** Para simular el hardware real, el simulador extrae el Índice y la Etiqueta (Tag) de cada dirección de memoria (representada en enteros de 64 bits) utilizando operadores de corrimiento a la derecha () y enmascaramiento lógico AND (`&`).

  Política de Reemplazo LRU (Least Recently Used): Se implementó un reloj global"que se incrementa en cada acceso a memoria. Cada línea de caché almacena una marca de tiempo. Ante un fallo en un conjunto lleno, el algoritmo busca linealmente y reemplaza la vía con la marca de tiempo más baja.

2.1. Definición y explicación de Algoritmos y estructuras

En el presente proyecto se optó por el uso de diferentes tipos de estructuras de datos y algoritmos para la representación y conceptualización de la forma en que la memoria caché gestiona sus datos e interactúa con distintas direcciones de memoria.

La estructura que representa la línea de la caché sería:

```
struct LineaCache {
    bool valida = false;
    uint64_t tag = 0;
    uint64_t contador_lru = 0;
};
```

Una línea es la unidad mínima de almacenamiento utilizada en la memoria caché; en ella se almacena un dato proveniente de la RAM. Funciona como un casillero.

La variable `valida` es utilizada para validar la disponibilidad de la casilla, se usa más que nada para los algoritmos de búsqueda y reemplazo, como cuando ocurren misses. La variable `contador_lru` es la marca de tiempo para la política de reemplazo.

El siguiente es `tag`, es el identificador único que utilizan los datos en el programa. Representa los bits más significativos (a la izquierda) de la dirección de memoria, los cuales se obtienen mediante desplazamientos de bits que se explican más adelante en el algoritmo principal del presente proyecto.

Las vías y los conjuntos: Un conjunto de líneas actúa como una red de casilleros que almacenan datos independientes.

```
struct ConjuntoCache{
    vector<LineaCache> lineas;
};
```

`ConjuntoCache` contiene el vector de líneas, donde el tamaño de dicho vector (la cantidad de líneas por conjunto) representa el número de vías de la memoria caché.

¿Cómo se decodifican los bits de la dirección? La dirección de memoria apunta a nivel de un byte individual, pero la caché no trae bytes sueltos, trae bloques enteros (el tamaño de línea).

Paso 1: Cortar el .ºffset"(Ignorar los bits de abajo)

```
uint64_t direccion_desplazada = direccion >> bits_offset;
```

El operador bits_offset empuja todos los bits hacia la derecha, botando por la borda los bits del offset. Ya nos libramos de ellos.

Paso 2: Aislar el Ïndex"(El número de fila exacta) Una vez que quitamos el offset, los siguientes bits de la derecha son el Index (el número de la fila o conjunto al que debemos ir). Pero claro, después del Index todavía quedan los bits del Tag hacia la izquierda. Si usaras el número completo, te saldrías del tamaño de tu vector y daría un error de memoria (Segmentation Fault). Para evitarlo, se usa una máscara de bits (`&`).

```
uint64_t index = (direccion >> bits_offset) & mascara_index;
```

¿Qué hace la operación `& mascara_index`? Funciona como un filtro de aduana. Deja pasar únicamente los bits que coinciden con el índice y apaga (vuelve cero) todo lo demás hacia la izquierda.

Paso 3: El salto directo al vector (`conjuntos[index]`) El resultado de esa operación matemática (index) es simplemente un número entero normal y corriente (por ejemplo, 5, 12 o 128). Como tu vector principal de conjuntos fue creado con un tamaño exacto (por ejemplo, 256 conjuntos), ese número de índice encaja perfectamente como la posición del vector:

```
ConjuntoCache& conjunto = conjuntos[index];
```

Si el cálculo dio 5, el programa hace `conjuntos[5]` y salta de manera instantánea a la fila 5 de tu matriz de vectores, ignorando por completo las demás filas.

Algoritmo principal

```
// PASO 1: Descomponer la dirección de memoria
1.1. Calcular el INDEX:
     index = (direccion >> bits_offset) & mascara_index
1.2. Calcular el TAG:
     tag = direccion >> (bits_offset + bits_index)

// PASO 2: Ir a la fila correspondiente en la matriz de conjuntos
2.1. Seleccionar el conjunto exacto:
     conjunto_actual = vector_conjuntos[index]

// PASO 3: Buscar si hay un HIT (Acierto)
3.1. Recorrer cada vía (línea) dentro de 'conjunto_actual':
     PARA CADA linea EN conjunto_actual.lineas HACER
          SI (linea.valida == VERDADERO) Y (linea.tag == tag) ENTONCES
                // ¡Lo encontramos! Es un Hit
                hits = hits + 1
                reloj_global = reloj_global + 1
                linea.contador_lru = reloj_global
                RETORNAR  // Fin del proceso para esta dirección
          FIN SI
     FIN PARA

// PASO 4: Si el bucle terminó y no retornó, ocurrió un MISS (Fallo)
4.1. Registrar el fallo:
     misses = misses + 1
     indice_reemplazo = -1
4.2. Buscar si hay una vía libre (vacía) en el conjunto:
     PARA i DESDE 0 HASTA (tamaño_vias - 1) HACER
          SI (conjunto_actual.lineas[i].valida == FALSO) ENTONCES
                indice_reemplazo = i
                ROMPER BUCLE
          FIN SI
     FIN PARA
4.3. Si el conjunto está lleno (no hay vías libres), aplicar LRU:
     SI (indice_reemplazo == -1) ENTONCES
          lru_mas_viejo = conjunto_actual.lineas[0].contador_lru
          indice_reemplazo = 0
          PARA i DESDE 1 HASTA (tamaño_vias - 1) HACER
                SI (conjunto_actual.lineas[i].contador_lru < lru_mas_viejo) ENTONCES
                     lru_mas_viejo = conjunto_actual.lineas[i].contador_lru
                     indice_reemplazo = i
                FIN SI
          FIN PARA
     FIN SI

// PASO 5: Rellenar la vía elegida (ya sea libre o la que expulsó el LRU)
5.1. Actualizar los datos de la línea:
     reloj_global = reloj_global + 1
```

```
conjunto_actual.lineas[indice_reemplazo].valida = VERDADERO
conjunto_actual.lineas[indice_reemplazo].tag = tag
conjunto_actual.lineas[indice_reemplazo].contador_lru = reloj_global
```

3. Metodología de la Prueba

Para evaluar el comportamiento de la memoria caché bajo distintas configuraciones de diseño, se sometió el simulador (configurado con una capacidad total de **32 KB** y asociatividad de **4 vías**) a un archivo de trazas (`trace_grande.txt`) estructurado con miles de accesos que combinan recorridos secuenciales locales y saltos dinámicos en memoria.

Se evaluaron tres tamaños de línea de caché distintos:

- **16 bytes**
- **64 bytes**
- **128 bytes**

La evaluación se midió a través de tres métricas fundamentales:

1. **Tasa de Aciertos (*Hit Rate*):** Porcentaje de accesos encontrados directamente en la caché.
2. **Tasa de Fallos (*Miss Rate*):** Porcentaje de accesos que requirieron buscar el bloque en niveles inferiores de memoria.
3. **Tiempo Medio de Acceso a Memoria (*AMAT*):** Métrica de rendimiento calculada mediante la fórmula estándar:

$$AMAT = Tiempodeacierto + (MissRate \times Penalizacinporfallo)$$

4. Resultados Obtenidos

En la Tabla 1 se sintetizan los datos recolectados tras la ejecución en lote sobre la traza de prueba.

Cuadro 1: Tabla Comparativa de Rendimiento del Simulador.

Tamaño de Línea	Hit Rate (%)	Miss Rate (%)	AMAT (ciclos)
16 bytes	67.24	32.76	33.76
64 bytes	84.50	15.50	16.50
128 bytes	87.67	12.33	13.33

5. Discusión y Análisis Técnico

5.1. Impacto de la Localidad Espacial

Los resultados reflejan una clara correlación positiva entre el tamaño de la línea de caché y la tasa de aciertos (*Hit Rate*). Al pasar de una línea de 16 bytes a una de 64 bytes, el *Hit Rate* aumenta de forma notable en un $+\mathbf{17.26\,\%}$, reduciendo el *Miss Rate* a la mitad.

Esto ocurre porque las trazas de programas reales exhiben **localidad espacial**: si un programa accede a una dirección de memoria, es altamente probable que en el futuro inmediato acceda a posiciones contiguas (por ejemplo, al recorrer arreglos o ejecutar instrucciones secuenciales). Con líneas de mayor tamaño, un solo fallo (*cold miss*) carga un bloque de datos más amplio que aprovecha los accesos subsiguientes sin incurrir en nuevos fallos.

5.2. Rendimientos Marginales (De 64 bytes a 128 bytes)

Aunque el paso de 16 a 64 bytes ofrece un salto drástico en rendimiento, al ampliar la línea de 64 a 128 bytes el incremento en el *Hit Rate* es más moderado ($+\mathbf{3.17\,\%}$, pasando de 84.50 % a 87.67

Este comportamiento ilustra el principio de rendimientos decrecientes en bloques demasiado grandes: si las líneas son excesivamente grandes (128 bytes o más), se corre el riesgo de sufrir **contaminación de caché** (*cache pollution*), trayendo datos innecesarios que desplazan información útil antes de que sea reutilizada, especialmente si el programa contiene saltos frecuentes o no mantiene una localidad estrictamente secuencial.

5.3. Comportamiento del AMAT (*Average Memory Access Time*)

El tiempo medio de acceso a memoria muestra una mejora sustancial a medida que se incrementa el tamaño del bloque, descendiendo desde los **33.76 ciclos** (con 16 bytes) hasta los **13.33 ciclos** (con 128 bytes).

Dado que el AMAT es directamente proporcional al *Miss Rate*, la reducción drástica de los fallos compensa con creces el costo asociado de transferir bloques más grandes desde la memoria principal, validando que una organización de bloque intermedia-alta (como 64 o 128 bytes) resulta óptima para cargas de trabajo con alta localidad espacial en arquitecturas de 32 KB.

6. Conclusiones

1. **Configuración Óptima:** Para el patrón de carga simulado, el tamaño de línea de **64 bytes** representa un punto de equilibrio excelente entre el aprovechamiento de la localidad espacial y la eficiencia de ocupación del espacio en la caché de 32 KB.

2. **Efectividad del Algoritmo LRU y Coherencia:** La implementación del algoritmo de reemplazo LRU en conjunto con la estructura de conjuntos asociativos de 4 vías manejó correctamente los conflictos de direccionamiento, garantizando métricas estables y consistentes durante la ejecución en lote.